From Conditional Safety to Local Proofs for Reactive Programs

Frédéric Dabrowski
Univ.Orléans, INSA Centre Val de Loire, LIFO EA 4022, Orléans, France
frederic.dabrowski@univ-orleans.fr

Abstract

This paper studies conditional safety properties of reactive programs over infinite streams: every admissible input stream should produce an execution satisfying a safety specification. The challenge is that the property is global in time, whereas the proof objects used in practice must remain local and compatible with deductive reasoning over programs with unbounded state. We present a semantic decomposition of this problem based on an explicit product between the program and two finite safety monitors representing assumptions and guarantees. From this product, we derive dangerous local steps, generated semantic clauses, and relational Hoare triples over reactive ticks. The main theorem states that, under explicit well-formedness hypotheses on the program model, validity of all generated triples suffices for conditional safety. The development also proves relative-completeness results: dangerous-step triples are semantically justified by global correctness, and the remaining coherence triples are justified under the semantic assumptions they are meant to encode. The paper gives a rigorous account of the reactive semantics, the clause-to-triple discipline, and the proof structure that reduces a global safety violation to a contradiction with a valid local triple. Concrete validation backends are orthogonal to this mathematical core and are discussed only as later refinements.

1 Introduction

The problem studied in this paper belongs simultaneously to three classical traditions. First, the target property is a *safety property* over infinite traces in the standard sense of the safety/liveness partition [1]. The temporal source language currently used by Kairos belongs to the lineage initiated by Pnueli's temporal logic of programs [2]; more precisely, the tool uses a safety fragment of LTL that can be compiled to finite safety automata [17, 18]. Second, the programs of interest are synchronous reactive programs evaluated on streams, in the style of Lustre and of the synchronous approach to reactive systems [5, 6, 14]. Third, the intended local proof objects are relational Hoare triples, so the deductive background is that of axiomatic semantics and relative completeness in the tradition of Hoare and Cook [3, 4, 19]. A convincing account of Kairos must therefore make explicit how these three viewpoints meet.

Synchronous verification has long relied on stream semantics, observers, contracts, and theorem proving. In Lustre-like settings, one reasons about reactive components that evolve in lockstep on infinite streams, either through observers and synchronous contracts or through proof-oriented semantic encodings [5, 6, 14, 10, 11, 12, 13]. In parallel, another line of work compiles temporal specifications into local program proof objects that can be discharged by deductive backends, as in Aorai or CaFE for C programs [8, 7, 9]. Kairos lies at the intersection of these two traditions: it is synchronous and reactive in the sense of stream-based semantics, but it also seeks a reduction from global temporal safety to local Hoare-style reasoning over ticks. This combination immediately raises the methodological question that drives the paper: what is the right semantic interface between a stream-level safety property and transition-level deductive reasoning?

The setting considered here is conditional safety over reactive executions. A program runs under an environment hypothesis on input streams, and its correctness is expressed by a safety

guarantee on the induced input/output trace. The user may also provide node invariants that capture additional facts about executions. The resulting statement is neither a plain state invariant nor a mere operational contract: it is a theorem of the form "admissible inputs imply safe executions," where both admissibility and safety are understood as properties of infinite traces and where the underlying program semantics remains synchronous and stream-based.

The main obstacle is not merely temporal expressiveness. The programs of interest manipulate non-bounded data domains: memories, inputs, and outputs are not assumed finite-state. This rules out a direct explicit-state account of the whole verification problem. The core challenge is therefore to preserve the meaning of a global safety property while reducing its proof to local objects that remain compatible with deductive reasoning over an unbounded transition system. This is precisely the point where the two classical views become insufficient when taken in isolation: automata alone forget the unbounded program state, while local contracts alone do not explain why a global temporal theorem follows.

This question is easy to underestimate. If one reasons only with finite temporal monitors, one forgets the unbounded program state. If one reasons only with Hoare-style local contracts, one loses the global temporal meaning of the specification. A paper at the level expected by POPL or PLDI must therefore make explicit not only the final theorem, but also the exact semantic path from global temporal safety to local deductive proof objects. This is the role of the present development.

The core proof objects of the paper are relational Hoare triples over reactive ticks. Each triple speaks about one program transition, a precondition on the current local tick context, and a postcondition on the successor context. The problem is then to generate the right preconditions and postconditions so that validity of these local triples really implies the original global safety statement. From the viewpoint of classical Hoare logic, Kairos is therefore not trying to replace local proof theory; it is trying to supply the correct local triples for a global temporal theorem. Relative-completeness questions thus arise twice in this setting: once for the reduction from global safety to generated triples, and once again for the deductive backend that must prove those triples.

To solve this problem, we exploit the fact that standard safety specification formalisms can be compiled to finite safety automata; this includes, in particular, the safety fragment of linear temporal logic used by the current Kairos toolchain [17, 18]. The paper therefore presents the mathematics directly in terms of assumption and guarantee safety automata. This allows us to define an explicit semantic product $program \times A \times G$, where A is the assumption automaton and G is the guarantee automaton. The product synchronizes, at each tick, a program transition, the reading of the current input by A, and the reading of the current observation by G. It provides the right semantic granularity for identifying dangerous local steps, extracting semantic clauses, and assembling the generated relational Hoare triples.

Contributions. The paper makes four contributions.

1. It defines a mathematical model of reactive programs over streams, together with a separate specification layer made of an assumption automaton, a guarantee automaton, and node invariants over local trace contexts.

2. It introduces the explicit product $program \times A \times G$ as the central semantic object from which dangerous steps and generated clauses are defined.

3. It formalizes a clause-to-triple discipline: generated semantic clauses are not the final proof objects; they are assembled into relational Hoare triples whose validity suffices for the conditional safety theorem.

4. It proves two theorem families. The first is a soundness theorem: validity of all generated triples implies conditional safety. The second is a family of relative-completeness theorems: under

global correctness and, where appropriate, semantic truth of the user-supplied annotations, the generated triples are valid and therefore are not spuriously false proof goals.

What is non-standard here. The novelty is not the mere use of automata, nor the mere use of deductive verification. The non-standard point is the way these two ingredients are tied together. The product $program \times A \times G$ is not introduced as an implementation device, nor as a model-checking state space to be exhaustively explored. It is the semantic object that justifies clause extraction, triple generation, and the reduction from a global bad execution to a local contradiction.

Scope of the result. The core theorem developed in this paper stops at semantic validity of generated relational Hoare triples. It does not prove the correctness of the backend grouping strategy, of the Why3 encoding, or of external prover integration. These are refinement layers that matter for the tool, but they are intentionally kept outside the core mathematical statement. The mathematical claims of the paper are therefore about generated triples and their semantic role, not about the full end-to-end trustworthiness of a particular solver stack.

The remainder of the paper is organized as follows. Section 2 defines reactive programs, their stream semantics, and the local trace contexts on which specifications are interpreted. Section 3 introduces deterministic safety automata and the interpretation of local predicates, including predicates with past. Section 4 presents the explicit product and its semantic role. Section 5 defines dangerous steps, local contexts, and the matching relation. Section 6 describes clause generation and triple assembly in a pseudo-algorithmic yet formal way. Section 7 reconstructs the proof chain from a global bad execution to the contradiction yielded by a valid generated triple. Section 8 discusses the connection between the mathematical presentation and the current implementation architecture.

2 Reactive Program and Stream Semantics

2.1 Reactive Program

A Kairos program is viewed as a stateful reactive system. The program state contains a control component and a memory component. Program transitions are not written as imperative assignments, but as logical relations between the current state, the current input, the current output, and the next state.

Definition 2.1 (Reactive Program)**.** *A reactive program P is given by:*

$$P = (S, M, I, O, T, s_0, m_0, \mathsf{Step}),$$

where:

- *S is the set of control states;*
- *M is the set of memory states;*
- *I is the set of inputs;*
- *O is the set of outputs;*
- *T is a set of transition identifiers;*
- *$s_0 \in S$ and $m_0 \in M$ are the initial conditions;*
- *$\mathsf{Step}(t, s, m, i, s', m', o)$ is a logical predicate, read as: transition t allows the passage from (s, m) to (s', m') under input i, producing output o.*

Definition 2.2 (Program Specification)**.** *The specification associated with P is a triple*

$$\Phi = (A, G, \mathsf{Inv}),$$

where:

- *A is a safety automaton over input streams;*
- *G is a safety automaton over reactive input/output traces;*
- $\mathsf{Inv} : S \to (\mathsf{TickCtx} \to \mathsf{Prop})$ *is an aggregated node invariant: for each control state s, $\mathsf{Inv}(s)$ is a property over tick contexts compatible with the execution.*

Remark 2.3. *The key point is the distinction between the* program *P, which contains only its syntax and step semantics, and the* specification *$\Phi = (A, G, \mathsf{Inv})$, which contains the external hypotheses, the guarantee, and node formulas over the execution trace. Kairos clauses therefore do not arise from an "enriched program" object, but from the coupling of P and Φ.*

2.2 Determinism and Step Relation

To avoid unnecessary overhead, we work in the deterministic setting targeted by Kairos: at each tick, the program selects a unique transition, and the automata also select a unique outgoing edge.

Definition 2.4 (Deterministic Program Step)**.** *Assume a functional step relation such that, for every (s, m, i), there exists a unique triple $(t, (s', m'), o)$ satisfying*

$$((s, m), i) \xrightarrow{t}_P ((s', m'), o).$$

This relation is compatible with Step*, namely:*

$$((s, m), i) \xrightarrow{t}_P ((s', m'), o) \Longrightarrow \mathsf{Step}(t, s, m, i, s', m', o).$$

Remark 2.5 (Totality Assumption on Program Ticks)**.** *The abstract model used in this paper assumes that every instant is total: from every current configuration and input valuation, the program computes a next control state, a next memory, and an output valuation. This is stronger than what a concrete implementation language may guarantee by construction. In particular, a concrete program may block because of an invalid operation, or fail to emit a value on some output signal. Such situations are intentionally abstracted away from the mathematical core and must therefore be ruled out by separate well-formedness conditions when one applies the model to actual Kairos programs.*

Remark 2.6 (Exhaustiveness of Transition Filtering)**.** *Another practical source of non-totality is the absence of any active transition for the current control state, memory, and input valuation. In the abstract model, this case is excluded: the filtering of transitions by current state and guard satisfaction is assumed to be exhaustive on the executions of interest. In practice, this should be guaranteed structurally, for instance by requiring exhaustive guarded cases or by inserting an explicit default branch at the source-language level. Introducing a semantic sink state inside the abstract program model would require specifying its observable output, which is not neutral; for this reason, the present paper treats exhaustiveness as a well-formedness requirement on concrete programs rather than as part of the core semantics.*

2.3 Stream Semantics

A local transition is not enough: one must define the execution of the program over an infinite input stream. This is where stream semantics comes in.

Definition 2.7 (Stream)**.** *For every set X, we write*

$$X^\omega := \mathbb{N} \to X$$

for the set of infinite streams over X.

Definition 2.8 (Configuration and Observation)**.** *We write*

$$\mathsf{Cfg} := S \times M, \qquad \mathsf{Obs} := I \times O.$$

Definition 2.9 (Admissible Initial Memories)**.** *We assume a predicate*

$$\mathsf{InitMem} \subseteq M$$

of admissible initial memories. The program starts in the distinguished control state s_0, but the initial memory is not required to be a single known value: any $m_0 \in \mathsf{InitMem}$ is allowed.

Definition 2.10 (Local Tick Context)**.** *We define*

$$\mathsf{TickCtx} := \mathbb{N} \times \mathsf{Cfg} \times I \times O \times \mathsf{Cfg}.$$

An element

$$(k, (s, m), i, o, (s', m')) \in \mathsf{TickCtx}$$

is called a local tick context*. It represents the smallest execution window around tick k: current configuration, current input, current output, and next configuration.*

Definition 2.11 (Trace-Derived Local Context)**.** *Fix an admissible initial memory $m_0 \in \mathsf{InitMem}$ and an input stream $u \in I^\omega$, and let $(s_{m_0,u}, m_{m_0,u}, o_{m_0,u})$ be the execution generated by P from m_0 on u. We write*

$$\mathsf{ctx}_{m_0,u}(k) := \big(k,\ (s_{m_0,u}(k), m_{m_0,u}(k)),\ u(k),\ o_{m_0,u}(k),\ (s_{m_0,u}(k+1), m_{m_0,u}(k+1))\big) \in \mathsf{TickCtx},$$

that is, the context obtained at tick k from the current configuration, the current input, the current output, and the next configuration of the execution.

Notation conventions. Fix an admissible initial memory $m_0 \in \mathsf{InitMem}$ and an input stream $u \in I^\omega$. The program P then generates an execution on (m_0, u), namely the sequences of control states, memories, and outputs produced tick after tick by the transition relation of P. We systematically write

$$s_{m_0,u} : \mathbb{N} \to S, \qquad m_{m_0,u} : \mathbb{N} \to M, \qquad o_{m_0,u} : \mathbb{N} \to O$$

for these three components of the execution of P from m_0 on u. To keep formulas readable, we omit the index m_0 when it is fixed by context and simply write s_u, m_u, o_u, $\mathsf{ctx}(u, k)$, and $\mathsf{run}(u)$. When we need to make the initial memory explicit, we write $\mathsf{ctx}_{m_0,u}(k)$ and $\mathsf{run}_{m_0}(u)$. Thus s_0 denotes the distinguished initial control state of the program, $m_0 \in \mathsf{InitMem}$ an admissible initial memory, $s_u(k)$ and $m_u(k)$ the control state and memory reached at tick k under input u, and $o_u(k)$ the output produced at tick k. For an input $u \in I^\omega$, execution therefore yields a sequence of configurations $(s_u(k), m_u(k))$, a sequence of outputs $o_u(k)$, and a reactive trace $\mathsf{run}(u)(k) = (u(k), o_u(k))$.

Primary definition. In this document, stream semantics is defined pointwise first.

Definition 2.12 (Stream Semantics)**.** *For every admissible initial memory* $m_0 \in \mathsf{InitMem}$ *and every input stream* $u \in I^\omega$*, we define the sequences* (s_u, m_u, o_u) *by:*

$$(s_u(0), m_u(0)) = (s_0, m_0),$$

and, for every $k \in \mathbb{N}$*, by the unique deterministic step*

$$((s_u(k), m_u(k)), u(k)) \xrightarrow{t_k}_P ((s_u(k+1), m_u(k+1)), o_u(k)).$$

The reactive trace is then defined by

$$\mathsf{run}(u)(k) := (u(k), o_u(k)).$$

Remark 2.13 (Coinductive Reading)**.** *This pointwise definition corresponds to the standard coinductive reading of a guarded stream equation: the head of the stream is given by the first program step, and the tail is obtained by reapplying the same rule to the input suffix and the next configuration. We use this reading as intuition, but the primary presentation in the paper is intentionally the operational definition above.*

3 Safety Automata and Predicate Interpretation

3.1 Safety Automata

The specification $\Phi = (A, G, \mathsf{Inv})$ contains three components: an assumption automaton A, a guarantee automaton G, and a node invariant Inv. We begin by fixing the general semantics of A and G.

A safety automaton reads an infinite word and rejects as soon as it reaches a *bad* state. In Kairos, edge labels are also written as logical relations, not as commands.

Definition 3.1 (Deterministic Safety Automaton)**.** *A deterministic safety automaton over an observation domain* X *is a tuple*

$$\mathcal{A} = (Q, q_0, bad, E, \mathsf{src}, \mathsf{dst}, \mathsf{Lab}, \mathsf{select}),$$

where:

- Q *is the set of states;*
- $q_0 \in Q$ *is the initial state;*
- $bad \in Q$ *is the violation state;*
- E *is a set of edges;*
- $\mathsf{src}, \mathsf{dst} : E \to Q$ *give the source and target;*
- $\mathsf{Lab}(e) : X \to \mathsf{Prop}$ *is the logical edge predicate;*
- $\mathsf{select}(q, x)$ *selects the unique outgoing edge from* q *whose label holds on* x*.*

Definition 3.2 (Run and Acceptance Condition)**.** *If* $w \in X^\omega$*, the execution of* $\mathcal{A}$ *on* w *is defined by*

$$q_{\mathcal{A},w}(0) = q_0, \qquad q_{\mathcal{A},w}(k+1) = \mathsf{dst}(\mathsf{select}(q_{\mathcal{A},w}(k), w(k))).$$

The word w *is accepted as safe if*

$$\forall k \in \mathbb{N}, \qquad q_{\mathcal{A},w}(k) \neq bad.$$

3.2 Interpretation of Local Predicates

The formulas used in A, G, and Inv are interpreted over $\mathsf{ctx}_u(k)$ contexts, not directly over the raw memory.

Definition 3.3 (Variables and Current Evaluation)**.** *Assume a set* Var *of observable program variables, together with a current-value function*

$$\mathsf{val}_{now} : \mathsf{TickCtx} \times \mathsf{Var} \to \mathsf{Val}$$

that reads the current value of each variable in the tick context. A free variable x in a formula therefore means the current value of x in the context, provided that x belongs to the domain of allowed observable variables.

Remark 3.4 (Abstract History Interface)**.** *The abstract model does not require a distinguished syntax of past operators. It only assumes that labels and invariants are predicates over local trace contexts and may therefore depend on suitable functions of the past. Concrete constructs such as prev or pre_k should be read as examples of how one may instantiate this abstract interface, not as primitive constituents of the core model.*

Definition 3.5 (Example Instantiation of History)**.** *If $x \in \mathsf{Var}$, one possible instantiation interprets $prev(x)$ at tick $k > 0$ by*

$$\mathsf{eval}_{\mathsf{ctx}_u(k)}(prev(x)) := \mathsf{val}_{now}(\mathsf{ctx}_u(k-1), x).$$

More generally, $pre_k(x)$ is interpreted by reading at tick $k - d$, whenever it is defined. Variables not placed under a past operator remain interpreted by the current valuation $\mathsf{val}_{now}(\mathsf{ctx}_u(k), x)$.

Remark 3.6. *The user-level intuition is that one may write formulas over current variables and, in a concrete language instance, over derived history operators such as prev. The abstract semantics only fixes how such formulas are interpreted through trace contexts. The finite encoding through auxiliary variables only appears later, on the implementation side.*

3.3 Node Invariant

Definition 3.7 (Node Invariant)**.** *A node invariant is a function*

$$\mathsf{Inv} : S \to (\mathsf{TickCtx} \to \mathsf{Prop}).$$

Thus, for each control state s, $\mathsf{Inv}(s)$ is a single property over tick contexts. If the user provides several elementary invariants for the same state, they are aggregated by conjunction into this single property.

Remark 3.8. *This matches the current Rocq presentation, where the node invariant is indexed by the state and then evaluated over a trace context. Semantically, this is cleaner than taking a primitive collection of distinct properties.*

3.4 Example: `resettable_delay`

The running example is a resettable one-tick delay. It has two control states

$$S = \{Init, Run\},$$

with memory $M = \mathbb{Z}$, input domain

$$I = \mathbb{B} \times \mathbb{Z},$$

where the two input components are written $reset$ and x, output $y \in O = \mathbb{Z}$, and the following four relational transitions:

$$\mathsf{Step}(t^{rst}_{init}, Init, m, (true, x), Run, m', y) :\Longleftrightarrow y = 0 \wedge m' = 0,$$
$$\mathsf{Step}(t^{keep}_{init}, Init, m, (false, x), Run, m', y) :\Longleftrightarrow y = 0 \wedge m' = x.$$

and, in the steady state,

$$\mathsf{Step}(t^{rst}_{run}, Run, m, (true, x), Run, m', y) :\Longleftrightarrow y = 0 \wedge m' = 0,$$
$$\mathsf{Step}(t^{keep}_{run}, Run, m, (false, x), Run, m', y) :\Longleftrightarrow y = m \wedge m' = x.$$

Intuitively, the node behaves as follows. If $reset$ is true, it outputs 0 and clears its memory. Otherwise, it behaves like an ordinary one-tick delay: it outputs the current memory and stores the current sample x. We use this example as the running case study because it remains compact while exhibiting a genuine interaction between input assumptions, guarantee clauses, and user invariants.

A natural user annotation is the aggregated node invariant

$$\mathsf{Inv}(Run)(\mathsf{ctx}_u(k)) :\Longleftrightarrow \big(prev(reset) \Rightarrow m = 0\big) \ \wedge \ \big(\neg prev(reset) \Rightarrow m = prev(x)\big),$$

where m denotes the unique memory cell. Since $M = \mathbb{Z}$ in this example, memory is reduced to a single integer register. The invariant says that, in state Run, the memory records either the reset value 0 or the previous sample, depending on whether the preceding tick asserted $reset$. Unfolded on the execution:

$$\psi_{rdelay}(k) :\Longleftrightarrow \begin{cases} s_u(k) = Run \Rightarrow \\ \qquad \big(prev(reset) \Rightarrow m_u(k) = 0\big) \wedge \\ \qquad \big(\neg prev(reset) \Rightarrow m_u(k) = prev(x)\big), \qquad (k \geq 1). \end{cases}$$

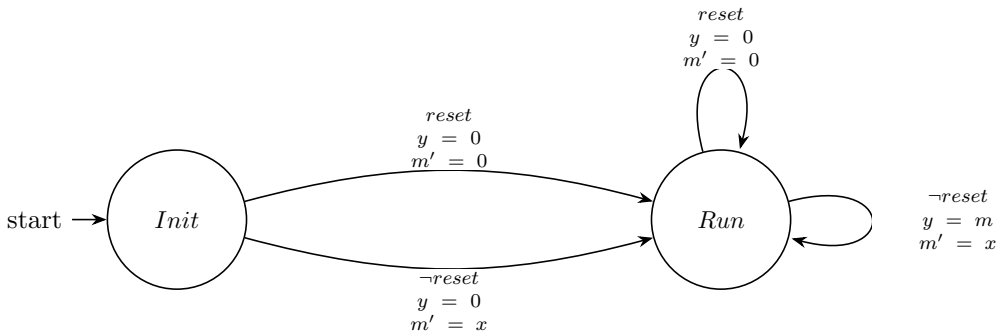

Figure 1: Program automaton for `resettable_delay`.

This program is simple enough to be understood operationally, but rich enough to require both assumption-side and invariant-side reasoning in the local proof objects.

4 Specification, Observations, and Guarantees

4.1 From Alphabet Words to Semantic Observations

Automaton labels do not necessarily range over a finite syntactic alphabet. They may instead be read as predicates over semantic observations.

Definition 4.1 (Observation Domains)**.** *In Kairos, one uses in practice $X_A = I$ for the assumption automaton A, $X_G = I \times O$ for the guarantee automaton G, and more generally a tick context derived from the trace when labels mention history.*

Remark 4.2. *The presence of past atoms such as $prev(x)$ does not change the nature of the automaton. It only means that the label is evaluated on the trace through a local context $\mathsf{ctx}_u(k)$ rich enough to interpret the past. In the implementation, this reading is then compiled through finite auxiliary variables; these variables are not part of the source semantics.*

4.2 Running Example: Assumption and Guarantee Automata

For the running `resettable_delay` example, we consider the following conditional safety specification:

$$A := G(\mathsf{ResetOk}),$$

and

$$G := G(\mathsf{GRst}) \ \wedge \ X\,G(\mathsf{GAfterRst}) \ \wedge \ X\,G(\mathsf{GDelay}).$$

where the three local predicates are

$$\mathsf{ResetOk} := (\neg reset \vee x = 0),$$

$$\mathsf{GRst} := (reset \Rightarrow y = x),$$

$$\mathsf{GAfterRst} := (\neg reset \wedge prev(reset) \Rightarrow y = 0),$$

$$\mathsf{GDelay} := (\neg reset \wedge \neg prev(reset) \Rightarrow y = prev(x)).$$

The assumption says that a reset command is only issued together with the value $x = 0$. The guarantee has three branches: on a reset tick, the output must agree with the reset value; on the first non-reset tick after a reset, the output must still be 0; and on ordinary non-reset operation, the output must equal the previous sample.

The assumption automaton has only two states:

$$Asm_{ok}, \qquad Asm_{bad}.$$

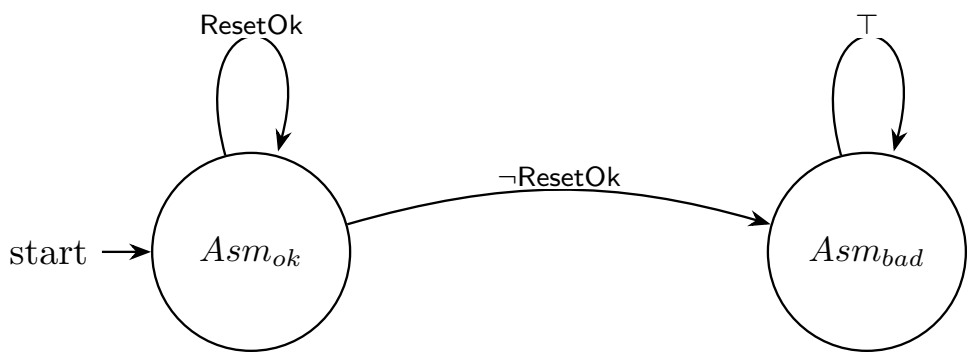

Figure 2: Assumption automaton for `resettable_delay`.

The guarantee automaton keeps track of whether the previous tick was a reset:

$$Mon_{init}, \qquad Mon_{prevR}, \qquad Mon_{prevN}, \qquad Mon_{bad}.$$

Unlabeled complements are written as *otherwise*: from each non-bad state, any valuation not covered by the explicit outgoing cases leads to Mon_{bad}.

Pedagogical reading of the example. This single example will be reused throughout the paper. At the program level, `resettable_delay` is still small; at the specification level, it combines a non-trivial input-side safety assumption, a guarantee with distinct reset and non-reset branches, and a user invariant that explains the ordinary delayed-data behavior. It is therefore complex enough to show the role of assumptions, guarantees, user invariants, dangerous steps, generated clauses, and generated triples in a single coherent running example.

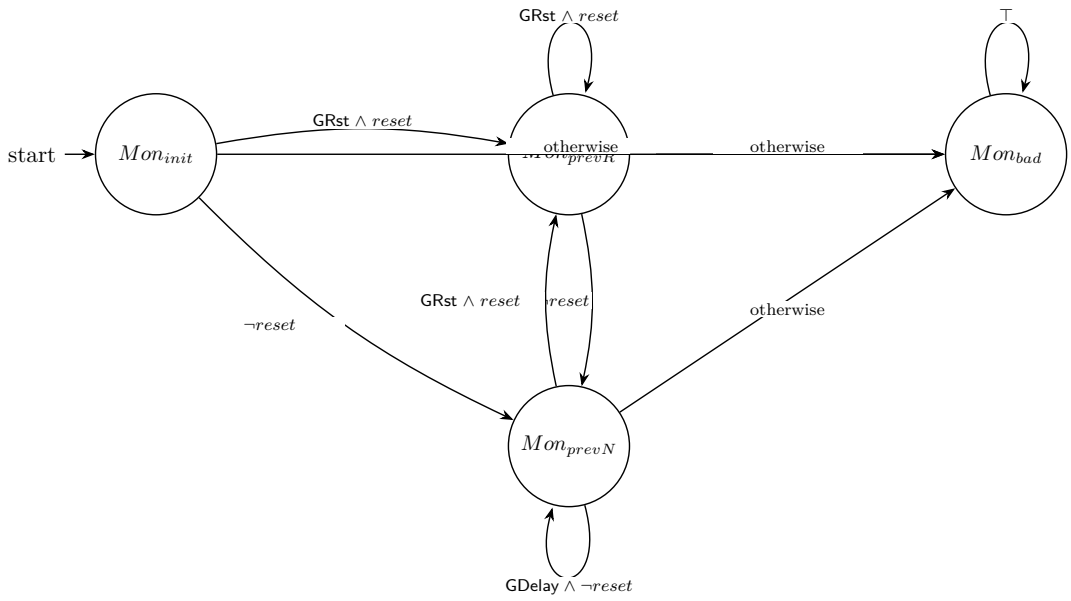

Figure 3: Guarantee automaton for `resettable_delay`.

5 Explicit Product $program \times A \times G$

5.1 Intuition

The explicit product is not meant to artificially "glue together" three graphs. It represents the following semantic fact: at each tick k, the program reads $i(k)$ and produces $o(k)$, A reads $i(k)$, and G reads $(i(k), o(k))$. There is therefore a *unique synchronized step* of the product.

5.2 Product States and Steps

Definition 5.1 (Program State and Product State)**.** *We write*

$$\mathsf{PState} := S.$$

A product state is a triple

$$\mathsf{ProdState} := S \times Q_A \times Q_G.$$

Remark 5.2. *This is indeed the* finite *component of the product. Memory, input, and output do not belong to the product state in the current Rocq formalization; they appear at the level of the concrete step and the local context.*

Definition 5.3 (Product Run)**.** *For an input stream u, we define:*

$$\mathsf{rps}(u, k) := \big(s_u(k),\, q_A(k),\, q_G(k)\big),$$

where q_A is the run of A on u, and q_G is the run of G on $\mathsf{run}(u)$.

Definition 5.4 (Initial Product State)**.** *The initial product state is*

$$\mathsf{init_product_state} := (s_0, q_{A,0}, q_{G,0}),$$

where $q_{A,0}$ and $q_{G,0}$ are the initial states of the automata A and G.

Definition 5.5 (Product Step)**.** *A product step p is an object of the form*

$$p = \big((s, q_A, q_G),\, m,\, i,\, t,\, e_A,\, e_G\big)$$

such that:

1. *the concrete program step satisfies*

$$((s,m),i) \xrightarrow{t}_P ((s',m'),o);$$

2. $\mathsf{src}_A(e_A) = q_A$ *and* $\mathsf{Lab}_A(e_A)(i) = \mathsf{true}$ *and* $\mathsf{dst}_A(e_A) = q'_A$*;*

3. $\mathsf{src}_G(e_G) = q_G$ *and* $\mathsf{Lab}_G(e_G)(i,o) = \mathsf{true}$ *and* $\mathsf{dst}_G(e_G) = q'_G$*.*

The target of the step is then (s', q'_A, q'_G)*.*

Remark 5.6. *The first point of this definition can also be read as*

$$((s,m),i) \xrightarrow{t}_P ((s',m'),o),$$

which makes the synchronization between the program step, the reading of A *on* i*, and the reading of* G *on* (i,o) *more visible, without making memory part of the finite product state.*

Remark 5.7. *Everything is written here as a logical relation:*

$$\mathsf{Step}(t,s,m,i,s',m',o), \qquad \mathsf{Lab}_A(e_A)(i), \qquad \mathsf{Lab}_G(e_G)(i,o).$$

We deliberately avoid imperative notations such as $m := i$*, which suit the code but not the mathematical model.*

5.3 Why This Product Is the Right Object

The explicit product plays three roles: it gives a common semantics to the program and the automata, it makes it possible to talk locally about situations in which G is about to be violated, and it provides a unique basis for the Rocq proof and for the product IR in the current implementation.

In particular, the product explains why a concrete tick is always described by a minimal quadruple of information:

$$\text{product source}, \qquad \text{input}, \qquad \text{output}, \qquad \text{product target}.$$

5.4 Example: Relevant Subgraph of `resettable_delay`

In `resettable_delay`, the most relevant product states are

$$(Init, Asm_{ok}, Mon_{init}),$$
$$(Run, Asm_{ok}, Mon_{prevR}),$$
$$(Run, Asm_{ok}, Mon_{prevN}),$$
$$(Run, Asm_{ok}, Mon_{bad}).$$

The initial tick leads either to Mon_{prevR} or to Mon_{prevN}, depending on the value of *reset*. Afterwards, two classes of dangerous transitions matter: reset-labeled dangerous steps, which should be excluded with the help of the assumption predicate $\mathsf{ResetOk}$, and non-reset dangerous steps from Mon_{prevN}, which should be excluded with the help of the user invariant describing the delayed sample.

6 Dangerous Steps and Local Obligations

6.1 Dangerous Step

The central notion of Kairos is now easy to define.

Definition 6.1 (Dangerous Step)**.** *A product step*

$$p : (s, q_A, q_G) \longrightarrow (s', q'_A, q'_G)$$

is dangerous *if*

$$q'_A \neq bad_A \qquad \textit{and} \qquad q'_G = bad_G.$$

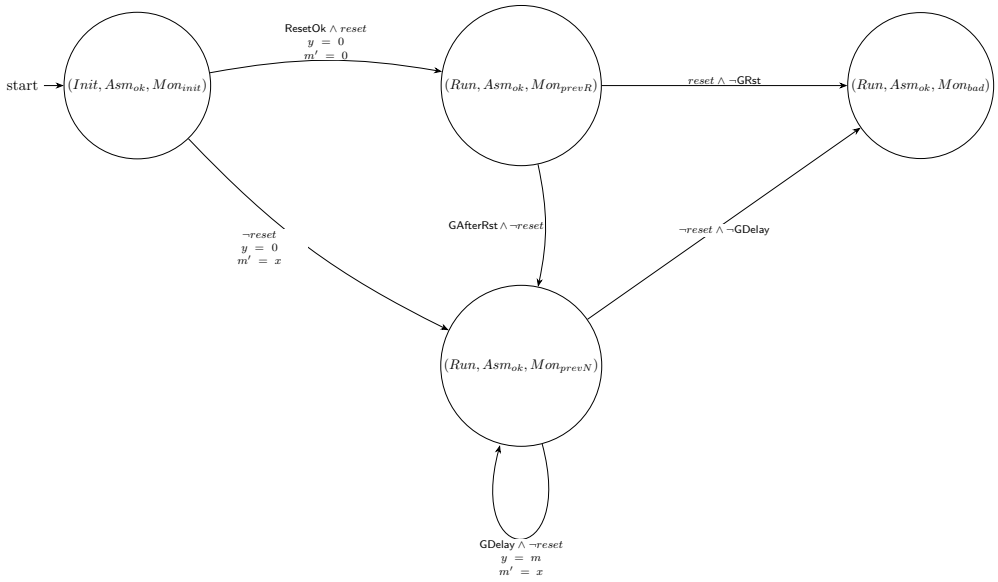

Figure 4: Relevant product subgraph for `resettable_delay`.

Intuition. This is not just any step to a bad state. One must distinguish two cases: if A itself goes to bad_A, then one leaves the admissible input domain and this is not the main objective of the conditional theorem; if A remains admissible but G goes to bad_G, then an admissible input is enough to produce a guarantee violation. This second situation is the one that must be ruled out locally.

Connection to the global semantics. If $\mathsf{AvoidG}(\mathsf{run}(u))$ is false, there exists a first index k such that $q_G(k+1) = bad_G$. Tick k then realizes a dangerous step. This observation is what allows one to move from a global violation over an infinite word to a local witness at a single tick.

6.2 Local Context and Matching Relation

Why introduce a local context? Obligation generation is static: it ranges over abstract product steps. Actual execution is dynamic: at each tick it realizes a concrete program transition and advances A and G. To connect these two levels, one must summarize the concrete tick by a local object, and then state when this object realizes a given abstract step.

Concrete tick versus abstract step. The *concrete tick* at rank k is the step actually observed in an execution under an input stream u: current configuration, current input, current output, and next configuration. The *abstract product step* is a purely static object, enumerated by the generator, which describes a possible transition of the triple $program \times A \times G$. Obligations are generated over abstract steps and then validated over all concrete ticks.

Definition 6.2 (Factored Local Context)**.** *We continue to write* $\mathsf{ctx}(u,k)$ *for the trace-derived local context* $\mathsf{ctx}_u(k)$ *introduced earlier. Thus,*

$$\mathsf{ctx}(u,k) = \big(k, (s_u(k), m_u(k)), u(k), o_u(k), (s_u(k+1), m_u(k+1))\big).$$

The automaton components attached to the same tick are not stored inside $\mathsf{TickCtx}$*; they are recovered separately from the product run* $\mathsf{rps}(u,k) = (s_u(k), q_A(k), q_G(k))$ *and* $\mathsf{rps}(u,k+1)$*.*

Remark 6.3. *This matches the underlying Rocq structure exactly:* `StepCtx` *carries only the concrete program tick, while the product-state components are tracked separately by the product run and by the realized product step.*

Definition 6.4 (Matching a Step)**.** *Let*

$$p = ((s, q_A, q_G), m, i, t, e_A, e_G)$$

be an abstract product step. We say that a local context ctx matches the program part of p*, and we write* $\mathsf{Match}(\mathsf{ctx}, p)$*, if:*

1. *the current program state stored in* ctx *is* s*;*
2. *the current memory stored in* ctx *is* m*;*
3. *the current input stored in* ctx *is* i*;*
4. *the selected program transition at this tick is* t*, equivalently* t *is enabled from* (s, m) *under* i*.*

Remark 6.5. *This is the primitive matching relation of the Rocq kernel (`ctx_matches_ps`). The automaton components are handled separately:*

- *the source automaton states are recovered from the product run* $\mathsf{rps}(u, k)$*;*
- *the target automaton states are recovered from the well-formedness of* p *and from the realized program transition.*

This decomposition is important: the elementary `NoBad` clause is only a clause over TickCtx*; the stronger automaton facts used to discharge it are supplied by coherence triples.*

6.3 Obligation Associated with a Dangerous Step

Definition 6.6 (Elementary Obligation)**.** *To every dangerous product step* p*, we associate the local formula*

$$\mathsf{obl}_p(\mathsf{ctx}) := \neg\mathsf{Match}(\mathsf{ctx}, p).$$

This form explains the exact intuition of the clause:

> "at the current tick, it is impossible to simultaneously realize the chosen program transition, the chosen assumption edge, and the guarantee edge leading to bad_G".

6.4 Detailed Example: `resettable_delay`

What the generator sees. The generator first explores the reachable part of the explicit product. In the steady phase of `resettable_delay`, the two important source states are

$$(Run, Asm_{ok}, Mon_{prevR}) \qquad \text{and} \qquad (Run, Asm_{ok}, Mon_{prevN}).$$

This already indicates that the guarantee proof splits into two semantic regimes: the regime immediately following a reset, and the ordinary delayed data regime.

Two representative dangerous steps. The running example has two representative dangerous families.

Dangerous reset step. From $(Run, Asm_{ok}, Mon_{prevR})$, consider a step combining:

1. the reset program transition t^{rst}_{run}, whose relation is

$$y = 0 \wedge m' = 0;$$

2. the assumption edge

$$Asm_{ok} \xrightarrow{\mathsf{ResetOk}} Asm_{ok};$$

3. the dangerous guarantee edge

$$Mon_{prevR} \xrightarrow{reset \wedge \neg \mathsf{GRst}} Mon_{bad}.$$

This dangerous step is relevant because it remains admissible for A, yet would violate the reset branch of the guarantee.

Dangerous non-reset step. From $(Run, Asm_{ok}, Mon_{prevN})$, consider a step combining:

1. the non-reset program transition t_{run}^{keep}, whose relation is

$$y = m \wedge m' = x;$$

2. the same assumption edge, still satisfied because $reset$ is false;

3. the dangerous guarantee edge

$$Mon_{prevN} \xrightarrow{\neg reset \wedge \neg \mathsf{GDelay}} Mon_{bad}.$$

This second dangerous family corresponds to the ordinary delayed-data branch of the specification.

The generated clauses. Each dangerous step yields a local clause:

$$\mathsf{obl}_{rst}(\mathsf{ctx}) := \neg \mathsf{Match}(\mathsf{ctx}, p_{rst}^{bad}), \qquad \mathsf{obl}_{keep}(\mathsf{ctx}) := \neg \mathsf{Match}(\mathsf{ctx}, p_{keep}^{bad}).$$

These clauses only forbid the program part of the two dangerous matchings. The full contradiction is recovered one level higher, inside the generated `NoBad` triples.

How the reset contradiction is obtained. For p_{rst}^{bad}, the `NoBad` triple combines:

1. the matching fact for the reset transition;

2. the automaton-support fact attached to $(Run, Asm_{ok}, Mon_{prevR})$;

3. the assumption-side fact $\mathsf{ResetOk}$.

The program relation gives $y = 0$, while the assumption support gives $x = 0$ on reset ticks. The dangerous guarantee edge requires $y \neq x$. Hence the triple precondition forces both

$$y = x \qquad \text{and} \qquad y \neq x,$$

which yields contradiction. This dangerous family is therefore ruled out primarily by the assumption side.

How the non-reset contradiction is obtained. For p_{keep}^{bad}, the `NoBad` triple uses:

1. the matching fact for the non-reset transition;

2. the user invariant at state Run;

3. the automaton-support fact attached to $(Run, Asm_{ok}, Mon_{prevN})$.

Here the program relation gives

$$y = m \qquad \text{and} \qquad m' = x,$$

while the invariant implies, in the Mon_{prevN} regime,

$$m = prev(x).$$

The dangerous guarantee edge requires

$$\neg reset \wedge \neg \mathsf{GDelay}.$$

Together, these facts force both

$$y = prev(x) \qquad \text{and} \qquad y \neq prev(x),$$

hence contradiction. This dangerous family is therefore ruled out primarily by the user-invariant side.

Why this example is better. The two dangerous branches play different proof roles:

- one branch is excluded by assumption support;
- the other branch is excluded by user-invariant support.

This is exactly the kind of interaction that the paper needs to exhibit in a single running example.

7 Pseudo-Algorithmic Generation of Clauses and Triples

7.1 Why a Procedural Description Is Necessary

It is not enough to enumerate clause families. To understand the Kairos architecture, one must explain how the tool constructs them from the product. The right view is the following:

1. explore reachable product states;
2. enumerate locally realizable product steps;
3. classify each step;
4. generate, according to this class, the appropriate clauses.

Immediate Instantiation on `resettable_delay`. This procedure already yields something very concrete:

1. the relevant reachable states are

$$(Init, Asm_{ok}, Mon_{init}) \quad \text{and} \quad (Run, Asm_{ok}, Mon_{prevR}), \quad (Run, Asm_{ok}, Mon_{prevN});$$

2. from these source states, one enumerates two representative dangerous families: a reset-dangerous one and a non-reset-dangerous one;
3. both are classified as "objective no bad";
4. one adds the local safety clause

$$\mathsf{obl}_{rst}(\mathsf{ctx}) = \neg\mathsf{Match}(\mathsf{ctx}, p^{bad}_{rst}) \qquad \text{or} \qquad \mathsf{obl}_{keep}(\mathsf{ctx}) = \neg\mathsf{Match}(\mathsf{ctx}, p^{bad}_{keep}).$$

The general procedural view is therefore not an extra layer: it describes exactly what the running `resettable_delay` case already does.

7.2 General Scheme

Input. The generator receives:

- a program P;
- a specification $\Phi = (A, G, \mathsf{Inv})$.

Output. It produces a set of semantic clauses, each equipped with:

- an abstract origin;
- a state or step of the product;
- a future backend projection.

7.3 Relational Hoare Triples Built from Generated Clauses

The core proof does not validate isolated clauses. It validates relational Hoare triples built from them.

Definition 7.1 (Generated Automaton-Support Predicate)**.** *We write*

$$\mathsf{Support} : \mathsf{ProdState} \to (\mathsf{TickCtx} \to \mathsf{Prop})$$

for the generated coherence predicate attached to product states. Intuitively, $\mathsf{Support}(x)$ *collects the automaton-derived local facts that are later used to justify transition preconditions while the product is at state* x*.*

Definition 7.2 (Initial Context)**.** *We write*

$$\mathsf{InitCtx}(\mathsf{ctx}) :\Longleftrightarrow \exists m_0 \in \mathsf{InitMem}, \exists u,\ \mathsf{ctx} = \mathsf{ctx}_{m_0,u}(0).$$

Definition 7.3 (Transition Relation on Local Contexts)**.** *Let* $t \in T$*. We define the semantic relation of* t *on local contexts by*

$$\begin{aligned}
&\mathsf{TransRel}_t(\mathsf{ctx}, \mathsf{ctx}') :\Longleftrightarrow \\
&\quad \exists m_0 \in \mathsf{InitMem}, \exists u \in I^\omega, \exists k \in \mathbb{N}, \\
&\qquad \mathsf{ctx} = \mathsf{ctx}_{m_0,u}(k) \ \wedge\ \mathsf{ctx}' = \mathsf{ctx}_{m_0,u}(k+1) \ \wedge \\
&\qquad ((s_u(k), m_u(k)), u(k)) \xrightarrow{t}_P ((s_u(k+1), m_u(k+1)), o_u(k)).
\end{aligned}$$

Definition 7.4 (Relational Hoare Triple)**.** *A relational Hoare triple is a tuple*

$$ht = (\mathsf{target}, \mathsf{Pre}, \mathsf{Post}, \mathsf{origin}, \mathsf{clause}),$$

where target *is either* Init *or* $\mathsf{Step}(t)$ *for some* $t \in T$*,* $\mathsf{Pre}, \mathsf{Post} : \mathsf{TickCtx} \to \mathsf{Prop}$*,* origin *records the conceptual category of the triple, and* clause *is the semantic clause carried by the triple.*

Definition 7.5 (Validity of a Relational Hoare Triple)**.** *The triple* ht *is valid if:*

$$\begin{aligned}
&\textit{if } \mathsf{target} = \mathsf{Init}, && \forall \mathsf{ctx},\ \mathsf{InitCtx}(\mathsf{ctx}) \wedge \mathsf{Pre}(\mathsf{ctx}) \Rightarrow \mathsf{Post}(\mathsf{ctx}); \\
&\textit{if } \mathsf{target} = \mathsf{Step}(t), && \forall \mathsf{ctx}, \mathsf{ctx}',\ \mathsf{TransRel}_t(\mathsf{ctx}, \mathsf{ctx}') \wedge \mathsf{Pre}(\mathsf{ctx}) \Rightarrow \mathsf{Post}(\mathsf{ctx}').
\end{aligned}$$

Definition 7.6 (Canonical Generated Triples)**.** *For a product step* p*, we write* t_p *for its underlying program transition,* $\mathsf{source}(p)$ *for its source product state, and* $\mathsf{target}(p)$ *for its target product state. If the program-state components of these two product states are* s *and* s'*, we write them explicitly in the invariant clauses below.*

The core generation relation contains five canonical forms of triples:

1. an initial triple for the user invariant:

$$\mathsf{InitInvTriple} : \mathsf{Init}\ \{\top\}\ \{\mathsf{Inv}(s_0)\};$$

2. an initial triple for automaton support:

$$\mathsf{InitSuppTriple} : \mathsf{Init}\ \{\top\}\ \{\mathsf{Support}(\mathsf{init_product_state})\};$$

3. for each well-formed product step p, a propagation triple for the user invariant:

$$\mathsf{InvTriple}_p : \mathsf{Step}(t_p)\ \{\mathsf{Match}(\cdot, p) \wedge \mathsf{Inv}(s)\}\ \{\mathsf{Inv}(s')\};$$

4. for each well-formed product step p, a propagation triple for automaton support:

$$\mathsf{SuppTriple}_p : \mathsf{Step}(t_p)\ \{\mathsf{Match}(\cdot, p) \wedge \mathsf{Support}(\mathsf{source}(p))\}\ \{\mathsf{Support}(\mathsf{target}(p))\};$$

5. for each dangerous well-formed product step p, a safety triple:

$$\mathsf{NoBadTriple}_p : \mathsf{Step}(t_p)\ \{\mathsf{Match}(\cdot, p) \wedge \mathsf{Inv}(s) \wedge \mathsf{Support}(\mathsf{source}(p))\}\ \{\mathsf{false}\}.$$

This last triple matters semantically when p is dangerous: in that case, a valid $\mathsf{NoBadTriple}_p$ *rules out the concrete realization of p.*

Remark 7.7. *The paper writes these triples schematically. In Rocq, their exact definitions are the initial invariant triple, the initial automaton-support triple, the invariant propagation triple, the automaton-support propagation triple, and the dangerous-step safety triple. The point of the present presentation is to make their logical role explicit before discussing backend grouping.*

Definition 7.8 (Generated Clauses and Generated Triples)**.** *We write* $\mathsf{GeneratedBy}(o, c)$ *when the semantic clause c is generated with origin o, and* $\mathsf{Generated}(c)$ *when such an origin exists. Concretely, the core generation relation contains the two initial clauses for node invariants and automaton support; for each well-formed product step p, one user-invariant propagation clause and one automaton-support propagation clause; and, for each dangerous well-formed product step p, one objective clause* $\mathsf{NoBad}_p(\mathsf{ctx}) = \neg\mathsf{Match}(\mathsf{ctx}, p)$*. Similarly,* $\mathsf{GeneratedTriple}(ht)$ *means that ht is one of the five canonical triples listed above, generated either from the initial state or from a well-formed product step, with the restriction that* `NoBad` *triples are generated only for dangerous steps.*

7.4 Conceptual Algorithm

For each reachable product state $x = (s, q_A, q_G)$:

1. Add the initial/coherence clauses attached to x.
2. Add the user-invariant propagation clauses attached to x, for example

$$\begin{aligned}&\mathsf{InvObl}_x(\mathsf{ctx}) :=\\ &\quad (\mathsf{src_state}(\mathsf{ctx}) = s \wedge \mathsf{src_a}(\mathsf{ctx}) = q_A \wedge \mathsf{src_g}(\mathsf{ctx}) = q_G)\\ &\quad \Rightarrow (\mathsf{Inv}(s)(\mathsf{ctx}) \Rightarrow \mathsf{Shift}(\mathsf{Inv}(s'))(\mathsf{ctx})).\end{aligned}$$

3. Add the automaton-support clauses attached to x, that is, the formulas that justify the additional local automaton facts assumed by the backend.
4. **For each** product step p outgoing from x:

a. If the target of p falls into bad_A, only add the support clauses that are still required.

b. If p is dangerous, add the objective clause

$$\mathsf{ObjObl}_p(\mathsf{ctx}) := \neg\mathsf{Match}(\mathsf{ctx}, p).$$

c. If p is safe, only add the coherence and support clauses required for this step.

The point of this procedural presentation is that *NoBad clauses are not expected to be discharged in isolation.* The backend proves them inside transition-level Hoare bundles, under extra local assumptions, and those assumptions are themselves justified by generated coherence clauses. The local proof architecture therefore has one safety layer and one coherence layer:

1. Coherence/InitGoal: establish, at the initial tick, both the user-invariant facts and the automaton-support facts;
2. Coherence/Propagation: propagate, from one tick to the next, both the user-invariant facts and the automaton-support facts;
3. Safety/NoBad: exclude dangerous product steps under this coherence context.

In the current Rocq core, this architecture is reconstructed in two steps. First, the concrete generation relation produces semantic clauses. Second, these clauses are assembled into relational Hoare triples whose validity is stated against the relational semantics of program transitions. The concrete generation relation contains:

$$\mathsf{prod_clause}(ps)(\mathsf{ctx}) := \neg\mathsf{ctx_matches_ps}(\mathsf{ctx}, ps)$$

for the objective layer, a preservation-style clause for node invariants, and a preservation-style clause for automaton support. The Rocq kernel now turns such clauses into relational Hoare triples; the implementation further groups these triples by program transition before translating them to Why3-facing verification tasks. What is still more fine-grained on the implementation side is the internal distinction between monitor-compatibility support and state-aware assumption support.

The implementation now exposes both levels explicitly. At the conceptual level, generated clauses are summarized as NoBad, InitialGoal, UserInvariant, and AutomatonSupport. At the backend level, the taxonomy is finer and keeps separate families such as:

- `FamNoBad*`;
- `FamInitialCoherencyGoal`;
- `FamCoherency*`;
- `FamMonitorCompatibilityRequires`;
- `FamStateAwareAssumptionRequires`.

The finer families are projected onto the same four conceptual categories as the Rocq proof architecture. These four categories are a classification of *generated clauses*, not four independent proof phases.

7.5 Four Clause Categories

The generated semantic clauses fall into four conceptual categories.

NoBad. These are the genuine safety objectives. For a dangerous product step p, the semantic clause is:

$$\mathsf{NoBad}_p(\mathsf{ctx}) := \neg\mathsf{Match}(\mathsf{ctx}, p).$$

The backend does not normally send this clause in isolation to the external tool. The kernel first turns it into a relational Hoare triple whose postcondition is $\bot$ on the dangerous branch; the implementation then places that triple inside a transition-level proof bundle together with the coherence preconditions and postconditions needed to make it provable. These clauses are sufficient to conclude that the product never reaches bad_G, but they are not expected to be discharged without coherence clauses.

InitialGoal. These clauses provide the base case of the induction. They establish that the coherence context already holds at tick 0. Concretely, this includes:

- initialization of the user invariant attached to the initial node;
- initialization of the automaton-support facts attached to the initial product state.

In the implementation, they correspond to explicit initial coherence triples grouped in the same initial coherence bundle. They could equivalently be encoded by a ghost predecessor carrying $\top$, but keeping them explicit is simpler and clearer.

UserInvariant. These clauses justify the node invariants used as local preconditions. The implementation injects:

- $\mathsf{Inv}(\mathsf{src})$ in transition preconditions;
- $\mathsf{Shift}(\mathsf{Inv}(\mathsf{dst}))$ in transition postconditions.

So the predecessor proves the fact that the successor later consumes as its own precondition. In the Rocq kernel this is expressed by a relational Hoare triple whose postcondition talks about the successor context. This is one *subfamily* of the common coherence propagation layer. These clauses must be bundled together with automaton support clauses for the same transition, because each side may help prove the other.

AutomatonSupport. These clauses justify the automaton-derived facts added to transition preconditions. There are two important subcases in the implementation:

- monitor compatibility clauses: if the current monitor state is q, then one of the incoming guards leading to q must have held at the previous tick, after shifting;
- state-aware assumption/product support clauses: for a fixed program transition and current guarantee state, one of the locally compatible (A-guard $\wedge$ G-guard) disjuncts must hold.

This is the second *subfamily* of the common coherence propagation layer. Again, in the Rocq kernel these clauses feed relational Hoare triples whose postconditions speak about the successor context. They are not meant to live in proof objects separate from user invariants: they must be discharged in the same coherence bundles.

Transition-level user contracts remain visible in the backend as separate `transition_requires` and `transition_ensures` families. They do not belong to the four generated-proof categories above. They are part of the user specification surface, not clauses generated by the local product proof architecture itself.

Why the coherence/safety split is needed. The soundness point is the following:

> it is valid to enrich a local dangerous-step proof with additional preconditions only if generated clauses separately establish that these preconditions do hold on concrete executions.

Therefore:

1. Coherence/InitGoal provides the base case for both invariant facts and automaton-support facts;
2. Coherence/Propagation propagates both kinds of coherence facts through time, inside the same transition-level proof objects;
3. Safety/NoBad excludes the dangerous product steps under this common coherence context.

Remark 7.9. *The four labels* NoBad*,* InitialGoal*,* UserInvariant*, and* AutomatonSupport *are useful as a classification of generated clauses. However, the proof architecture is not a chain of four isolated layers:* UserInvariant *and* AutomatonSupport *must be bundled together inside the same coherence proof objects, both at initialization and at propagation time.*

8 Proof Structure

8.1 Global Assumptions of the Core Theorem

The final theorem of the core does not model an external checker. It is stated directly under two groups of hypotheses.

Program-model well-formedness. The program must satisfy the structural assumptions captured by the Rocq record `WellFormedProgramModel`:

- the selected program transition is enabled at every tick;
- the selected edges of A and G do start from the current automaton states;
- the labels of these selected edges are true of the current input and current observation;
- the initial states of A and G are not bad.

These assumptions express that the semantic model is total on the class of programs under consideration. They are complemented by the separate initial memory admissibility predicate InitMem, which characterizes the class of initial memories on which the theorem is stated.

Validation hypothesis. The second global hypothesis states that all generated relational Hoare triples are valid:

$$\forall ht,\ GeneratedTriple(ht) \Rightarrow TripleValid(ht).$$

Everything else is proved inside the core from this hypothesis.

What is proved in this section. The proof section establishes two result families. The soundness theorem states that validity of all generated triples implies the global conditional-safety property. The relative-completeness theorems go in the converse direction, but only to the extent justified by the semantics: `NoBad` triples are shown valid under global correctness, automaton-coherence triples are shown valid from the product semantics itself, and user-invariant triples are shown valid under the additional hypothesis that the user invariants are semantically true on admissible runs.

8.2 Step 1: Product Progression

Proposition 8.1 (Progress of the Explicit Product)**.** *For every admissible initial memory $m_0 \in \mathsf{InitMem}$, every input stream u, and every tick k, there exists a well-formed product step ps such that:*

1. *ps is the step realized at tick k;*
2. *the target of ps is exactly the product state reached at tick $k+1$.*

Proof. This is the product-progression theorem of the Rocq development. The proof is by explicit construction: the semantic selectors of the program, of A, and of G define a canonical step $\mathsf{product_select_at}(u, k)$. The well-formedness part follows from `WellFormedProgramModel`. The target computation is then checked by unfolding the definitions of the program step, of the automaton updates, and of the local observation sent to G. □

Proposition 8.2 (A Realized Step Induces the Transition Relation)**.** *Let u be an input stream, k a tick, and p a product step realized at tick k. Then the underlying program transition t_p satisfies*

$$\mathsf{TransRel}_{t_p}(\mathsf{ctx}(u, k), \mathsf{ctx}(u, k+1)).$$

Proof. This is the content of the Rocq lemma `transition_rel_of_realized_step`. Once p is known to be the step realized at tick k, the relation $\mathsf{TransRel}_{t_p}$ follows immediately from its definition. □

8.3 Step 2: From a Global Violation to a Dangerous Local Step

Proposition 8.3 (Global Violation Implies a Dangerous Tick)**.** *For every admissible initial memory $m_0 \in \mathsf{InitMem}$ and every input stream u, if $\mathsf{AvoidA}(u)$ and $\neg\mathsf{AvoidG}(\mathsf{run}_{m_0}(u))$, then there exists a tick k such that $bad_local_step(m_0, u, k)$.*

Proof. This is the global-to-local reduction theorem. Since $\neg\mathsf{AvoidG}(\mathsf{run}(u))$, there exists a first index j such that $q_G(j) = bad_G$. The initial state of G is not bad, hence $j = S\,k$ for some k. By Step 1, a concrete well-formed product step is realized at tick k; its target is the product state reached at tick $k+1$. The guarantee component of this target is therefore bad_G. Because $\mathsf{AvoidA}(u)$ holds, the assumption component of the same target is not bad_A. This is exactly the definition of a dangerous local step. □

8.4 Step 3: Coherence Facts Hold on Admissible Runs

Proposition 8.4 (Initial Coherence Facts)**.** *For every admissible initial memory $m_0 \in \mathsf{InitMem}$ and every admissible run on u, the two coherence facts needed by the local proof are true at the initial tick:*

1. *the user invariant attached to the initial program state holds;*
2. *the automaton-support fact attached to the initial product state holds.*

Proof. These are the two initial-coherence propositions of the Rocq development. Each one is obtained by applying the validity hypothesis to the corresponding generated initial triple. The precondition of these triples is the initial-context predicate, which holds by construction of $\mathsf{ctx}_{m_0,u}(0)$. Their postconditions are precisely the coherence facts to be established at tick 0. □

Proposition 8.5 (Propagation of Coherence Facts)**.** *Let $m_0 \in \mathsf{InitMem}$ and let u be an admissible input stream. Then for every tick k:*

1. *the user invariant of the current program state holds on $\mathsf{ctx}(u, k)$;*

2. the automaton-support fact of the current product state holds on $\mathsf{ctx}(u, k)$.

Proof. This is the content of the two run-propagation propositions in Rocq. Both proofs are inductions on k.

The base case is given by the previous proposition. For the induction step, the canonical realized product step at tick k yields the generated propagation triples `node_inv_triple ps` and `support_automaton_triple ps`. By the induction hypothesis, the corresponding preconditions hold on $\mathsf{ctx}(u, k)$. By validity of generated triples, together with the transition-relation proposition of Step 1, their postconditions hold on $\mathsf{ctx}(u, k+1)$. This establishes the two coherence facts at the successor tick. □

Proposition 8.6 (Combined Coherence Context)**.** *Let* $m_0 \in \mathsf{InitMem}$ *and let* u *be admissible. If a well-formed product step* ps *is realized at tick* k*, then the full coherence context required by the safety triple attached to* ps *holds on* $\mathsf{ctx}_{m_0,u}(k)$.

Proof. This is the combined-coherence proposition. It is an immediate combination of the two propagation results above, instantiated at the source program state and source product state of the realized step ps. □

8.5 Step 4: A Dangerous Tick Produces a Falsified Generated Clause

Proposition 8.7 (Clause Coverage of Dangerous Steps)**.** *For every admissible initial memory* $m_0 \in \mathsf{InitMem}$*, every input stream* u*, and every tick* k*, if* $bad_local_step(m_0, u, k)$*, then there exists a generated semantic clause* c *such that:*

$$Generated(c) \qquad and \qquad \neg c(\mathsf{ctx}_{m_0,u}(k)).$$

Proof. This is the clause-coverage theorem. By definition of *bad_local_step*, a dangerous tick comes with a well-formed realized product step ps. The core associates to ps the safety clause stating that the current concrete tick does not match this dangerous step. But a local lemma proves that the concrete context $\mathsf{ctx}(u, k)$ does match the realized step ps. Therefore this clause is falsified at $\mathsf{ctx}(u, k)$. □

8.6 Step 5: A Dangerous Tick Produces an Applicable `NoBad` Triple

Proposition 8.8 (Triple Coverage of Dangerous Steps)**.** *For every admissible initial memory* $m_0 \in \mathsf{InitMem}$*, every input stream* u*, and every tick* k*, if* $bad_local_step(m_0, u, k)$*, then there exist a product step* ps *and a generated triple* ht *such that:*

1. $ht =$ ***no_bad_triple*** ps*;*
2. *the precondition of* ht *holds on* $\mathsf{ctx}(u, k)$*;*
3. *the postcondition of* ht *is* false.

Proof. This is the triple-coverage theorem. Again, a dangerous tick provides a realized dangerous product step ps. The core generates from this step the triple `no_bad_triple ps`. Its postcondition is by construction the always-false clause `FalseClause`. Its precondition is the conjunction of three ingredients:

1. the matching condition saying that the current tick realizes ps;
2. the user-invariant fact required at the current source state;
3. the automaton-support fact required at the current source product state.

The first item comes from the realized-step matching lemma; the other two come from the combined coherence-context corollary of Step 3. Hence the precondition of `no_bad_triple ps` does hold on the dangerous tick. □

8.7 Step 6: Valid Triples Rule Out Global Violations

Theorem 8.9 (Conditional Correctness)**.** *Assume* `WellFormedProgramModel` *and assume that every generated triple is valid. Then, for every admissible initial memory* $m_0 \in \mathsf{InitMem}$ *and every input stream* u*,*

$$\mathsf{AvoidA}(u) \Rightarrow \mathsf{AvoidG}(\mathsf{run}_{m_0}(u)).$$

Proof. This is the main soundness theorem of the Rocq development. Fix an admissible initial memory $m_0 \in \mathsf{InitMem}$, an input stream u, and assume $\mathsf{AvoidA}(u)$. Suppose, for contradiction, that $\neg\mathsf{AvoidG}(\mathsf{run}_{m_0}(u))$. By Step 2, there exists a dangerous tick k. By Step 5, there exist a dangerous realized product step ps and a generated triple $ht =$ `no_bad_triple` ps such that:

1. the precondition of ht holds on $\mathsf{ctx}(u, k)$;
2. the postcondition of ht is false;
3. the transition of ps is exactly the transition realized at tick k.

Since ht is generated, the global validity hypothesis yields $TripleValid(ht)$. Instantiating this validity statement with the concrete tick k and the concrete successor context $\mathsf{ctx}(u, k+1)$, one obtains that the postcondition of ht must hold at the successor tick; Step 1 provides the required transition-relation premise. But that postcondition is false, a contradiction. Therefore no global violation of G can occur on an admissible input stream, and $\mathsf{AvoidG}(\mathsf{run}(u))$ follows. □

Remark 8.10. *The clause-level theorem* `generation_coverage` *and the triple-level theorem* `triple_generation_coverage` *play different roles. The former is the sharp semantic reduction from a dangerous local step to a falsified generated clause. The latter is the bridge actually used by the final proof: it shows that the same dangerous local step also yields a generated validatable object, namely a* `NoBad` *relational Hoare triple whose precondition is satisfied on the concrete tick.*

8.8 Relative Completeness Results

The main theorem above is a soundness result: validity of all generated triples implies conditional safety. The Rocq development also exposes three complementary *relative completeness* results. Their role is not to claim that every coherence fact is forced by the global specification alone. Rather, they show that the generation discipline does not introduce spurious local proof goals once one assumes the semantic facts that these goals are supposed to encode.

Theorem 8.11 (Relative Completeness of NoBad Triples)**.** *Assume* `WellFormedProgramModel`*. Assume furthermore that the program is globally correct in the following sense:*

$$\forall m_0 \in \mathsf{InitMem},\ \forall u, \qquad \mathsf{AvoidA}(u) \Rightarrow \mathsf{AvoidG}(\mathsf{run}_{m_0}(u)).$$

Then, for every dangerous well-formed product step p*, the generated triple* $\mathsf{NoBadTriple}_p$ *is valid on admissible runs.*

Proof. This is the relative-completeness theorem for NoBad triples. Consider an admissible run and a tick satisfying the precondition of $\mathsf{NoBadTriple}_p$. The coherence part of this precondition states directly that the source of p is the current concrete product state, so the matching fact can be promoted to a realized dangerous step at that tick. But a realized dangerous step contradicts global correctness: by the local-to- global lemma of Step 2, it would force a violation of G on an admissible run. Hence the precondition of a dangerous $\mathsf{NoBadTriple}_p$ can never hold on an admissible run, which is precisely the validity of the triple. □

Theorem 8.12 (Automaton-Support Completeness)**.** *Assume* `WellFormedProgramModel`*. Then the initial support triple is valid on admissible runs and, for every well-formed product step* p*, the generated support propagation triple* $\mathsf{SuppTriple}_p$ *is valid on admissible runs.*

Proof. This is the relative-completeness theorem for automaton support. For the initial triple, validity is immediate from the definition of the initial product state and of the initial concrete context. For a propagation triple, assume a concrete admissible run and a tick satisfying its precondition. The coherence part of this precondition says directly that the source of p is the actual current product state, so p is the realized step at that tick. The postcondition is then exactly the coherence statement for the successor product state at tick $k+1$, which follows from the semantics of $\mathsf{ctx}(u, k+1)$. □

Theorem 8.13 (Relative Completeness of User Invariants)**.** *Assume* `WellFormedProgramModel`*. Assume that user invariants are true on all admissible runs. Then the initial user-invariant triple is valid on admissible runs and, for every well-formed product step p, the generated user-invariant propagation triple* $\mathsf{InvTriple}_p$ *is valid on admissible runs.*

Proof. This is the relative-completeness theorem for user invariants. The initial case is immediate from the truth of user invariants at the initial tick. For a propagation triple, the matching part of the precondition identifies the control-state component of the target of p with the control state reached at the successor tick. The postcondition is therefore exactly the user invariant known to hold at that successor tick on admissible runs. □

Theorem 8.14 (Generated-Triples Completeness)**.** *Assume* `WellFormedProgramModel`*. Assume furthermore that the program is globally correct and that user invariants are true on all admissible runs. Then every generated triple is valid on admissible runs.*

Proof. This is the global relative-completeness theorem. The proof is a case analysis on the generation relation for triples. There are exactly five generated shapes:

1. the initial user-invariant triple;
2. the initial automaton-support triple;
3. a propagation user-invariant triple;
4. a propagation automaton-support triple;
5. a `NoBad` triple attached to a dangerous step.

The first and third cases are discharged by the user-invariant completeness theorem above. The second and fourth cases are discharged by the automaton- support completeness theorem above. The fifth case is discharged by the `NoBad` completeness theorem. Since every generated triple falls into exactly one of these cases, all generated triples are valid on admissible runs. □

Remark 8.15. *These relative-completeness statements should be read carefully. The* `NoBad` *result is a genuine completeness property for the safety reduction: if the program is globally correct, the generated dangerous-step triples are not spuriously false. The automaton-support part is now internal to the semantic product and therefore provable from the model itself. By contrast, the user-invariant part remains* relative *to semantic assumptions about the truth of the supplied invariants, which come from the specification rather than from the temporal reduction alone.*

9 Implementation Language and Why3 Backend

9.1 Source Language of the Implementation

The mathematical model introduced above is reflected in the current Kairos implementation through two layers that are kept conceptually distinct.

Program layer. A node contains input and output declarations, local and state variables, a finite set of control states with a distinguished initial state, and a transition relation described by guarded node transitions. This is the executable part of the source program.

Specification layer. The same node also carries assumptions, interpreted over input traces, guarantees, interpreted over reactive input/output traces, and state-related user invariants, interpreted over local trace contexts. In the implementation, this split is explicit: the program part provides the transition semantics, while the specification part provides the formulas from which automata and support clauses are derived.

9.2 From Detailed Clauses to Backend Code

The detailed semantic clauses defined in this paper are not sent to Why3 as raw $\neg\mathsf{Match}(\mathsf{ctx}, p)$ formulas over abstract product steps. They are first assembled into relational Hoare triples and then projected to a finite backend representation.

Step 1: product analysis. The implementation builds or explores the product $program \times A \times G$. This exploration computes reachable product states, realizable product steps, the classification of steps into safe, dangerous, inadmissible, or pruned cases, and the abstract local clauses attached to those states and steps.

Step 2: backend projection. These abstract clauses and triples are then compiled into an instrumented program. Automaton information is represented by finite backend variables such as `__aut_state`; references to past values are compiled through auxiliary variables such as `__pre_k...`; and local clauses become backend preconditions, postconditions, linking invariants, and transition-side formulas. The purpose of this projection is not to change the semantics, but to replace the explicit matching relation over product steps with formulas that Why3 can attach to concrete program declarations.

Step 3: Why3 generation. From this instrumented intermediate form, Kairos produces: an OBC-like textual form used for debugging and inspection, a Why3 abstract syntax tree, a rendered Why3 source file, and optional verification-condition and SMT dumps, one block per goal. At this stage, the triples of the paper have become Why3 verification conditions attached to functions, transitions, and auxiliary lemmas of the generated program [19].

9.3 Concrete Walkthrough on `resettable_delay`

The current toolchain can already be exercised on a concrete Kairos source version of the running example. Since this section is also meant to illustrate the concrete syntax of the implementation language, we reproduce the full node:

```
node resettable_delay(reset: int, x: int) returns (y: int)
contracts
  requires: G((reset = 0 or reset = 1) and (reset = 1 => x = 0));
  ensures: G((reset = 1) => (y = 0));
  ensures: X G((reset = 0 and prev reset = 1) => y = 0);
  ensures: X G((reset = 0 and prev reset = 0) => y = prev x);
locals
  m : int;
states
  Init(init), Run;
invariants
```

```
  in Run:
    (prev reset = 1 => m = 0) and
    (prev reset = 0 => m = prev x);
transitions
  Init:
    to Run when reset = 1 { y := 0; m := 0; }
    to Run when reset = 0 { y := 0; m := x; }
  Run:
    to Run when reset = 1 { y := 0; m := 0; }
    to Run when reset = 0 { y := m; m := x; }
end
```

The actual source used in the implementation repository is `resettable_delay.kairos` in the test inputs directory; the generated artifacts used below are produced directly from that file with the V2 CLI.

Residual automata produced by the tool. On this example, the current implementation prints the following residual states: on the assumption side, $A0$ and $A1$, where $A1$ is the inconsistent residual false; on the guarantee side, $G0$, $G1$, and $G2$, where $G1$ is the inconsistent residual false and $G2$ is the steady-state residual obtained after the first tick. This is the concrete manifestation of the usual safety-monitoring workflow: state predicates are first propositionalized into tick-local atoms, then compiled into residual temporal states, and finally rendered as finite safety automata.

Reachable product and generated clauses. The current product exploration reports five reachable states:

$$(Init, A0, G0), \quad (Run, A0, G1), \quad (Run, A0, G2), \quad (Run, A1, G1), \quad (Run, A1, G2).$$

For this concrete node, Kairos also reports 22 generated clauses, split as follows: 3 safety clauses and 19 coherence clauses; among coherence clauses, 1 is an initial goal and 18 are propagation clauses; among generated clause kinds, 6 come from user invariants and 12 from automaton support. This is already enough to illustrate the architecture of the paper on a real example: the source specification is compiled to residual automata, the product is explored, and the resulting dangerous or supportive situations are translated to local proof objects.

Instrumented OBC and generated Why3 code. The generated OBC-like form makes the backend projection explicit. For `resettable_delay`, the tool introduces the ghost fields `__aut_state`, `__pre_k1_x`, and `__pre_k1_reset`. Transition contracts then mention: compatibility facts about the monitor state, assumption-side support formulas, coherence formulas carrying the user invariant, and safety formulas excluding `NoBad` combinations. For instance, the concrete extracted OBC-like transition for the initial reset case contains the expected combination of compatibility, assumption-side support, a `NoBad` postcondition, and the shifted coherence fact:

```
transition Init -> Run when reset = 1 {
  require __aut_state = Aut0                          {Compatibility};
  require __aut_state = Aut0 -> ...                   {AssumeAutomaton};
  ensure  __aut_state = Aut0 -> not (...)             {Instrumentation};
  ensure  (reset = 1 -> m = 0) and (reset = 0 -> m = x) {Coherency};
  do
    y := 0;
    m := 0;
}
```

The corresponding Why3 file contains the same information in Why3 syntax: an enumerated type for monitor states, a record for program variables including ghost fields, a `step` procedure, and the projected preconditions and postconditions.

Validation status on the current backend. The extraction pipeline succeeds end-to-end on this example: automata, reachable product, clause map, OBC-like code, and Why3 source are all generated. On the current backend, the resulting Why3 development is discharged by Z3 when Why3 is run with the standard simplification and splitting transformations used for transition VCs:

```
tmpconf=$(mktemp /tmp/why3conf.XXXXXX)
why3 config --config "$tmpconf" detect
why3 -C "$tmpconf" prove \
  -a simplify_formula \
  -a eliminate_if_term \
  -a remove_unused \
  -a split_vc \
  -P z3 -t 30 resettable_delay.mlw
```

In our current environment, this command exits successfully on the generated file. This matters for the paper for two reasons: it confirms that the mathematical reduction can already be exercised on a concrete Kairos example through the implementation path, and it makes visible the exact boundary between the semantic proof objects of the paper and the backend proof obligations actually discharged by Why3.

9.4 What Is Actually Sent to Why3

The object sent to Why3 is therefore not the mathematical product itself, but a finite program annotated with the formulas that result from the projection above. Concretely, the generated Why3 file contains declarations for program states, memories, and auxiliary variables; one or more Why3 procedures encoding the program step relation; preconditions and postconditions representing the projected clauses; and additional formulas linking source variables, target variables, automaton states, and history variables.

From the point of view of the present paper, the essential invariant is the following: every backend verification condition must correspond to a semantic fact justified by the product analysis. In particular, dangerous product steps must give rise to formulas whose failure would exactly witness the local realization of such a step.

Additional structural side conditions. Because the abstract model assumes that every tick computes a successor state, a successor memory, and an output valuation, a practical Kairos backend may need companion checks or side conditions ensuring that the concrete program falls within this fragment. Such conditions concern, for instance, the absence of runtime blocking caused by invalid operations, the fact that each output signal is assigned a value at every instant, and the exhaustiveness of the transition filtering by current state and memory. These conditions do not belong to the abstract product core itself; they are prerequisites for applying the abstract semantics to the concrete source language.

9.5 What Kairos Retrieves from Why3

When Why3 is invoked, Kairos receives a collection of proof goals together with their statuses. For each goal, the relevant result is one of the standard Why3 outcomes: proved, invalid, unknown, timeout, or backend failure.

Meaning of a positive result. If a goal is proved, then the corresponding projected backend formula is valid in the logic handled by Why3 and its external provers. Under the intended correctness theorem of the paper, such a result is used as evidence that the associated local triple is valid, and therefore that its underlying clause or support condition holds. This also applies to any side conditions or companion checks used to guarantee that the concrete program belongs to the total fragment assumed by the model.

Meaning of a negative or inconclusive result. If a goal is reported invalid, unknown, timed out, or failed, the mathematical meaning is weaker than "the property is false". It means that Kairos did not obtain external validation for the projected formula. This may reveal: a real bug in the program or in the specification, an insufficient invariant or support clause, a weakness of the backend encoding, or a limitation of the automatic prover. Hence the result returned by Why3 must always be interpreted relative to the projection correctness assumptions of the overall method.

9.6 Why the Backend Still Matters

The explicit product remains the right semantic reference, but it is not the right final object to materialize exhaustively in generated Why3 code. The backend remains necessary in order to: keep the generated code finite and manageable, encode automaton and history information through ordinary program variables, and expose triples and their projected clauses in a format accepted by Why3 and SMT solvers. The methodological split is therefore the following: the product provides the semantic source of the clauses, while the backend packages them into transition-level triples and provides their finite proof-oriented encoding.

10 Related Work

The closest related work falls into three technical families. The first one reduces temporal properties of programs to local proof material checked by a deductive backend. The second one studies synchronous contracts, observers, and assume/guarantee reasoning over reactive systems. The third one compiles temporal properties into executable monitoring artifacts. Kairos borrows an essential idea from each family, but differs from all of them in the semantic role assigned to the explicit product $program \times A \times G$. This comparison matters because the contribution of Kairos is not merely another automata-based verifier: its main claim is that the explicit product is the right semantic object for transporting a global conditional safety property toward local deductive proof objects over a program that may manipulate unbounded data.

For clarity, the comparison should be read along two axes. The first axis is *what is proved*: a property of program traces, a contract over a synchronous transition system, or a runtime-monitoring obligation. The second axis is *how the proof is mediated*: through annotations attached to program points, through invariants over a synchronous transition system, or through an explicit semantic product between a program and temporal automata. Kairos is best understood as occupying the corner "reactive program theorem + explicit product + deductive local proof objects".

Another useful axis of comparison is finite-state versus unbounded-state reasoning. Classical temporal model checking is most natural when the system and the temporal monitor can be explored in one explicit transition system. The present setting is different: the automata are finite-state, but the program state space need not be. Kairos therefore combines finite safety automata with deductive transition reasoning instead of reducing the whole problem to explicit-state exploration. The contribution is thus not an explicit-state algorithm for a finite product, but a proof architecture in which the finite temporal component and the deductive program component meet at the level of product steps, generated clauses, and relational Hoare triples.

Classical background. The background notions used throughout the paper are classical. Safety is understood in the standard trace-theoretic sense [1]; the temporal source language belongs to the linear-time tradition inaugurated by Pnueli [2]; the local proof layer is phrased in the language of Hoare triples and therefore inherits the usual soundness/relative-completeness questions of axiomatic semantics [3, 4]; and the stream semantics of programs belongs to the synchronous approach to reactive systems [5, 6, 14]. Kairos should thus be read as a construction at the intersection of these established traditions, rather than as a competing foundational framework.

10.1 From Temporal Properties to Program Proof Objects

Aorai. Aorai [8, 7] is probably the closest neighbor to Kairos if one focuses on the idea of compiling temporal properties into local proof material attached to a program. In Aorai, LTL properties about C programs are translated into automata and then into ACSL-style annotations checked by the Frama-C toolchain. The key similarity with Kairos is the reduction from a global temporal property to finitely many local proof objects. The main difference is semantic. Aorai is centered on annotated C programs and temporal properties over program traces, whereas Kairos is centered on reactive traces with an explicit distinction between input assumptions and input/output guarantees. Kairos also makes the product $program \times A \times G$ explicit as a semantic object, instead of keeping the automaton/program synchronization mostly implicit in the generated annotations. This explicit product is not cosmetic: it is what lets Kairos state and prove the reduction from a global guarantee violation to a dangerous local step, which is the central proof idea of the present work.

CaFE. CaFE [9] extends the same general ecosystem to CaRet properties, which are tailored to nested calls and returns. This makes it especially relevant when temporal reasoning must account for call-stack structure. Its proximity to Kairos lies in the idea that temporal reasoning over programs can be reduced to local proof objects checked by an existing backend. Its difference from Kairos lies in the observed semantic domain: CaFE is organized around call/return structure of C executions, whereas Kairos is organized around reactive ticks, stateful stream semantics, and assume/guarantee safety on input/output traces. In particular, Kairos is not merely "Aorai for synchronous programs": its target property is conditional safety under input admissibility, and its core semantic object already combines program and automata state.

10.2 Assume/Guarantee and Synchronous Verification

AGREE. AGREE [10, 11] is close to Kairos on the logical structure of specifications. It emphasizes assumptions and guarantees attached to components, and it supports observer-style encodings of temporal patterns. This places it closer to Kairos than Aorai with respect to contract structure. However, AGREE works at an architectural and compositional level, not at the level of an explicit semantic product between a program automaton and two safety automata. Kairos differs by making the local product step itself the primary justification point for local clauses. AGREE is therefore close to Kairos on the shape of specifications, but not on the proof-theoretic role played by explicit automata/program synchronization.

Kind 2 and CoCoSpec. Kind 2 [12] and CoCoSpec [13] are also strong reference points for contract-based reasoning on synchronous systems. Kind 2 offers realizability checking and proof support for Lustre-style programs and contracts, while CoCoSpec provides a mode-aware contract language for reactive systems. These systems are close to Kairos in their emphasis on reactive contracts and explicit assumptions/guarantees. The main difference is again the central semantic object. Kairos builds its local proof story around the product between a program, an assumption automaton, and a guarantee automaton. Kind 2 and CoCoSpec are closer to invariant generation,

model checking, and contract reasoning directly over synchronous transition systems than to an explicit local matching relation between concrete ticks and abstract product steps. This is an important distinction: Kairos does not seek only an invariant-based proof over the source transition system, but a semantic bridge from stream-level safety to local product-step reasoning.

Lustre and theorem proving. Older work on Lustre and theorem proving [14] is also relevant as historical background. It already combines a reactive programming language with machine-checked reasoning and therefore shares with Kairos the ambition of giving a mathematical semantics that supports actual proofs. Kairos differs by organizing the reduction around safety automata and locally generated clauses and triples, rather than around direct theorem proving over the source transition system alone. In that sense, Kairos is closer to a proof-oriented compilation pipeline than to a direct embedding of the source semantics into a proof assistant.

10.3 Monitoring and Observer Compilation

Copilot. Copilot [16, 15] is an important point of comparison on the compilation side. It compiles temporal and stream-oriented specifications into executable monitors, typically generating hard real-time C code. This is close to Kairos in the sense that both approaches turn temporal specifications into finite operational artifacts. The main difference is that Copilot is first and foremost a runtime verification framework, whereas Kairos uses its compiled artifacts to generate proof objects for deductive verification. Copilot is particularly relevant as related work on monitor compilation, but it is further from Kairos than Aorai, AGREE, or Kind 2 on the proof-theoretic side.

Summary of the comparison. The literature already contains, separately, deductive encodings of temporal properties over programs, synchronous contract reasoning, and compilation of temporal monitors. What is rarer is a mathematically explicit account of how a global conditional safety theorem over reactive streams is reduced to local proof objects when the temporal layer is finite-state but the program itself is not. This is the niche filled by the present Kairos formulation.

10.4 Position of Kairos

Relative to these systems, Kairos combines four ingredients that are rarely present together in a single framework:

- a reactive program semantics on streams;
- an explicit input assumption automaton and an explicit guarantee automaton;
- a semantic product $program \times A \times G$ used as the source of local clauses;
- a proof-oriented reduction from global bad executions to falsified local safety clauses and then to contradictory valid triples.

This makes Kairos simultaneously close to annotation-based temporal program verification, to assume/guarantee reasoning for synchronous systems, and to the observer-compilation tradition. What distinguishes it is not simply the use of automata, nor simply the use of deductive verification, but the fact that the explicit product is the semantic source of generated clauses and triples, while the program itself is allowed to live over unbounded domains. Put differently, Kairos occupies a specific point in the design space: it inherits the temporal discipline of finite safety automata, the proof ambition of deductive program verification, and the contract structure of synchronous assume/guarantee reasoning, but it organizes them around an explicit product semantics rather than around a purely operational monitor or a purely invariant-oriented transition-system proof.

11 Discussion

11.1 Why One Must Constantly Return to the Product

A major source of confusion in this kind of document is to talk about clauses as if they were floating formulas. In reality, every Kairos clause must always be read as the projection of some information carried by the product: a reachable product state, a safe product step, a dangerous product step, or a user annotation reattached to that state or step. This discipline explains the editorial choice of the present text: it returns systematically to the program and to the product before discussing the backend.

11.2 Limits of the Present Note

This document does not claim to replace the detailed Rocq proof, the exhaustive description of the Why3/OBC backend, or a complete complexity study. Its aim is more focused: to provide a mathematical presentation that is complete and pedagogical enough to serve as a stable reference between the proof, the code, and the documentation.

12 Conclusion

The main claim of this paper is that conditional safety of reactive programs over unbounded domains can be reduced to local proof objects without losing its trace-theoretic meaning. The right central object for this reduction is the explicit product

$$\text{program} \times A \times G.$$

It makes the following simultaneously visible: the reactive semantics of the program on streams, the reading of assumptions and guarantees by safety automata, the notion of dangerous step, the generation of local clauses and triples, and the bridge both to the Rocq proof and to the external backend.

The main methodological lesson is therefore precise. In a high-quality account of Kairos, clauses should never be introduced as a mere list of local formulas. They must always appear as projections of the explicit product, and the actual proof objects exported to deductive verification must be relational Hoare triples assembled from those projections. This is what keeps the meaning of the final theorem stable while moving from stream-level safety to local proof objects.

Just as importantly, the paper clarifies the scope of what has and has not been proved. The core theorem concerns reactive semantics, generated clauses, generated relational triples, and their validity. It yields a soundness result for the reduction and a family of relative-completeness results that explain why the generated local proof obligations are not arbitrary. Backend grouping, Why3 encoding, and solver integration remain refinement layers. This separation is not a weakness of the presentation; it is part of the scientific contribution, because it identifies exactly which part of the architecture carries the semantic content of the correctness argument and which part belongs to implementation engineering.

The remaining research directions are therefore well delimited: strengthen the mathematical account of backend refinement, characterize more precisely the well-formed program fragment accepted by the implementation, and study how far the present clause-and-triple discipline can be pushed beyond the current deterministic safety setting. More generally, the paper suggests that explicit product semantics is a robust way to connect temporal safety, synchronous semantics, and local deductive proof in settings where the temporal layer is finite-state but the program state space is not.

References

[1] Bowen Alpern and Fred B. Schneider. Defining liveness. *Information Processing Letters*, 21(4):181–185, 1985.

[2] Amir Pnueli. The temporal logic of programs. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*, pages 46–57, 1977.

[3] C. A. R. Hoare. An axiomatic basis for computer programming. *Communications of the ACM*, 12(10):576–580, 1969.

[4] Stephen A. Cook. Soundness and completeness of an axiom system for program verification. *SIAM Journal on Computing*, 7(1):70–90, 1978.

[5] Nicolas Halbwachs, Paul Caspi, Pascal Raymond, and Daniel Pilaud. The synchronous dataflow programming language Lustre. *Proceedings of the IEEE*, 79(9):1305–1320, 1991.

[6] Albert Benveniste and Gérard Berry. The synchronous approach to reactive and real-time systems. *Proceedings of the IEEE*, 79(9):1270–1282, 1991.

[7] Julien Groslambert and Nicolas Stouls. Vérification de propriétés LTL sur des programmes C par génération d'annotations. In *Approches Formelles dans l'Assistance au Développement de Logiciels (AFADL)*, 2009.

[8] Nicolas Stouls. *Aoraï Plug-in Tutorial*. Frama-C manual, available from the Frama-C documentation site.

[9] Frama-C Team. *CaFE (CaRet Frama-C Extension)*. Frama-C plugin documentation page.

[10] John D. Backes, Andrew Gacek, Michael W. Whalen, Lee Pike, and others. *AGREE: Assume-Guarantee Reasoning Environment*. Technical report, Loonwerks, 2021.

[11] John D. Backes, Michael W. Whalen, Andrew Gacek, and John Komp. On implementing real-time specification patterns using observers. In *NASA Formal Methods*, 2016.

[12] Temesghen Kahsai, Andrew Gacek, Yeting Ge, and others. Realizability checking of contracts with Kind 2. In *NASA Formal Methods*, 2022.

[13] Curtis et al. CoCoSpec: a mode-aware contract language for reactive systems. In *Formal Methods in Computer-Aided Design*, 2016.

[14] Saddek Bensalem, Pascale Caspi, David Chemouil, and others. A methodology for proving control systems with Lustre and PVS. In *Proceedings of the 7th International Workshop on Higher Order Logic Theorem Proving and Its Applications*, 1994.

[15] Lee Pike, Alwyn Goodloe, Jeffrey Maddalon, and others. Copilot 3: monitoring autonomous systems. NASA technical memorandum, 2020.

[16] Copilot Language Project. *Copilot: a stream-based runtime verification framework*. Project repository and documentation.

[17] Orna Kupferman and Moshe Y. Vardi. Model checking of safety properties. In *Computer Aided Verification*, 1999.

[18] Paul Gastin and Denis Oddoux. Fast LTL to Büchi automata translation. In *Computer Aided Verification*, 2001.

[19] François Bobot, Jean-Christophe Filliâtre, Claude Marché, and Andrei Paskevich. Why3: shepherd your herd of provers. In *Boogie*, 2011.